



İSTANBUL SABAHATTIN ZAIM UNIVERSITY

BIM 439 DATA MINING

BITMATCH

Prepared by:
030721044 - Mustafa Zafer

Submission Date: January 5, 2026

Contents

1	Introduction	2
2	Literature Review	2
3	Framework Structure, Components and Architecture	2
3.1	Object Detection Strategy	3
3.2	Classification Strategy and Challenges	3
4	Software Usage	3
4.1	System Requirements	4
4.2	User Interface and Workflow	4
5	Algorithm Description of the Framework	5
5.1	Algorithm Workflow	5
6	Runtime Samples	5
6.1	Mobile App Usage (Flutter)	6
6.2	Request to the API	6
6.3	Displayed Output	6
7	Results and Discussion	6
7.1	Classification Model Analysis	6
7.2	Object Detection Model Analysis	7
7.3	General Discussion and Synthesis	7
8	Future Work	7

1 Introduction

Screws used in electronic devices often cause confusion because of their small size and similar shapes. To address this problem, we developed a mobile application. Especially with users with vision difficulties, its hard to choose correct bit.

Using wrong screwdriver may cause the damage of the screw head, or in worst case, it can cause a damage to electronic device. As a solution to this problem, we developed **BITMATCH** project by using Data Mining, Machine Learning and Image Processing technics.

In this project, we build an mobile app to automatically detect the screws in the user given photo and classify it by (PH,SL,T,PZ) its types. The architecture contains 2 models. First model is Object Detection model, and second one is Classification Model.

2 Literature Review

Computer Vision is commonly used in industrial systems for detecting and classifying objects. Many studies show that detecting small objects such as screws is more difficult than detecting large objects because of their small size and similar shapes. In addition, metal surfaces and different lighting conditions make this task harder.

In the literature, deep learning-based object detection models such as R-CNN and YOLO are widely used. YOLO is often preferred because it is fast and suitable for real-time and mobile applications. Previous studies also state that using data collected from real-world conditions improves model performance compared to using only controlled datasets.

3 Framework Structure, Components and Architecture

In this section, we will talk about the methods used, problems faced and solutions we made.

3.1 Object Detection Strategy

Initially, the WSDS dataset is used for develop an YOLO Object Detection Model, but in the real life tests and different conditions the model has failed.

In order to solve this problem, new data is collected and labeled by *labelImg* tool. With this new dataset, the new model is *fine-tuned* with the old model. This hybrid model had a significant performance increase.

3.2 Classification Strategy and Challenges

In classification problem, the problem solved stage by stage. :

1. **Initial (3 Classes):** Initial model has trained in 3 classes which is PH,SL and T and reaches great performance rate.
2. **Unstability Problem (PZ Class):** The Pozidriv screws added later on the dataset. But when the pozidriv screws are taken, but there were too many PZ images. So model memorized just the Pozidriv type of screws, and classify all of the test images as Pozidriv.
3. **Expanding the Dataset and Preprocessing:** In order to solve to unstability problem, new 100 images taken for every screw classes. For model to understand better the difference between PZ and PH type of screws, the images turned into Grayscale and added sharpening filter just before classification process. But this decision made it worse and model predicts everything as Pozidriv and Phillips.
4. **Data Cleaning and Final Solution :** In last step, we focused on the data quality and right model. We cleaned the blurry and bad quality images and make a new dataset with approximately 600 photos. And we changed the model we used to Yolo11m-cls from Yolo11s-cls. In this version of model, we reached high performance metrics.

4 Software Usage

In this section, we will talk about the BITMATCH system requirements, setup and how to use as an user. System divided into 2 layer as Mobile and Server-Side.

4.1 System Requirements

In order to perform, the user should have this requirements:

1. **Server-Side Requirements (Backend):** YOLO models works on the Python based server.

- **Programming Language:** Python 3.9 or above.
- **Libraries:** Ultralytics (for YOLO), FastAPI , OpenCV (for Image Processing), NumPy.

2. **Client-Side Requirements (Mobile Application):** For cross-platform support and high performance, we developed with Google **Flutter** framework.

- **Framework:** Flutter SDK (Dart Language).
- **Operating System:** Android 8.0 (Oreo) and above or iOS 12.0 or above.
- **Hardware Permissions:** In order to launch the app, you must give **Camera** permission and for the API connection you must have **network connection**.

4.2 User Interface and Workflow

Mobile app designed simply and user-friendly. The instructions of the app :

1. **Image Capture:** When the app launches, user press the camera button on main screen and take an image of an screw or picks and image from gallery.
2. **API Request:** The image sent to the Python server with HTTP POST request. In this stage, the image processed and turned into JPEG format.
3. **Processing:** Server takes the images and use it as an input of the Object Detection Model, find its bounding box and crops it. The cropped input is then applied to the filter. And then it goes to the Classification model.

4. **Result Display:** The results goes back to the mobile client by json format. Client draws a bounding box around the screw and says the which type of the screw it is.

5 Algorithm Description of the Framework

5.1 Algorithm Workflow

Step 1: Input

The raw image captured by the user through the mobile application is first processed on the client side. The EXIF data is read to fix rotation problems, and the image is converted into the correct orientation (portrait or landscape). After that, the image is encoded into a standard JPEG format and sent to the server.

Step 2: Object Detection:

The image received by the server is first passed to the Yolo8s Object Detection model. The model scans the image to detect screw .A confidence score is generated for each detection.Predictions with a confidence value below the defined threshold (0.20) are removed.

Step 3: Cropping Interested Zone:

Using the detected coordinates, only the region containing the screw is cropped from the original image. This step helps the classification model focus only on the screw head.

Step 4: Classification

The cropped screw image is given as input to the Yolo11m-cls Classification model. The model calculates probability scores for each screw type (PH, PZ, SL, T). The class with the highest probability is assigned as the final screw type.

6 Runtime Samples

This section demonstrates how the system is used on sample data through the mobile application.

6.1 Mobile App Usage (Flutter)

The user selects an image from the gallery or captures a new photo using the camera. Before sending the image to the backend, the application converts it into a clean JPEG and applies EXIF orientation correction to ensure consistent image rotation.

6.2 Request to the API

The mobile app sends the corrected JPEG image to the FastAPI endpoint `/predict` as a multipart/form-data POST request. Detection and classification parameters are provided via query parameters (e.g., `det_conf`, `det_iou`, `cls_imgsz`).

6.3 Displayed Output

The returned JSON response includes bounding boxes and classification results. The app visualizes bounding boxes on the image and lists each detection with confidence scores and Top-K classification predictions.

7 Results and Discussion

7.1 Classification Model Analysis

- **Training Performance:** Model, reached **%99.16 Top-1 Accuracy** in the train and *validation* datasets.
- **Real-World Test:** In order to test the (*robustness*) of the model, the new test images is created (50 images) and the model has reached **%85 Accuracy** this time.
- **Class-Based Analysis:**
 - **Slotted (SL):** Due to its geometric shape, this class reached %100 Precision ve Recall in this new dataset.
 - **Phillips (PH) vs. Pozidriv (PZ):** Its main difficulty for the model. For the PZ class, the Precision level is calculated as %67. The cause of this is the PH and PZ screws are very similar to each other.

7.2 Object Detection Model Analysis

Object Detection model is trained for 51 epochs. The results are like this:

- **mAP:** The model achieved a score of **96.3%** on the **mAP@50** (Mean Average Precision at 0.5 IoU) metric. This result shows that the model can detect screws almost without error at a 50% overlap threshold.
- **Localization Accuracy (mAP@50–95):** The **mAP@50–95** value, which represents the average performance across stricter IoU thresholds, was measured as **73.7%**. This result indicates that the model not only detects the presence of objects but also places the bounding boxes very accurately on the objects.
- **Precision and Recall:** At the end of training, the model reached values close to **0.89 Precision** and **0.91 Recall**.
- **Loss Analysis:** The *box_loss* and *cls_loss* values showed a stable decrease within the training process, while signs of overfitting is at a minimal level.

7.3 General Discussion and Synthesis

1. **Dataset Characteristics and Domain Shift:** While both the classification and detection models achieved high performance in the range of **96%–99%** on the validation set, a drop to around **85%** was observed in real-world tests (for classification). This situation known in the literature as **Domain Shift**, meaning that the training and validation datasets are very similar while real-world conditions different from the dataset.
2. **Morphological Similarity Issue:** The distinction between PH and PZ screws is a challenging problem for computer vision models.

8 Future Work

In this section, we will talk about what could be done more in this project.

- **Offline:** System requires network connection and Python-server to run. In future works, we are planning the trained model can be transform into TensorFlow format and can be run on the mobile app.
- **Scope Expansion:** System supports just 4 screw (PH,PZ,SL and T). In future stages, dataset will be expand and more screw types will be added.
- **Size Estimation:** Model is just predicting the type of the scope. In future works we will develop the size estimation (by using a coin or etc) with using comparing methods and predict the size of the screw.

References

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 779–788).
- [2] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO (Version 8.0)* [Software]. Available from <https://github.com/ultralytics/ultralytics>
- [3] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- [4] Google. (2017). *Flutter SDK* [Software]. Available from <https://flutter.dev>
- [5] Terven, J., & Cordova-Esparza, D. (2023). A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*, 5(4), 1680–1716.
- [6] WEEE Disassembly Screw Dataset (WDSD). Available from <https://vcl.iti.gr/dataset/weee-disassembly-screw-dataset/>
- [7] Tzutalin. (2015). *LabelImg* [Software]. Available from <https://github.com/tzutalin/labelImg>